

Meta Android Developer Professional Certificate - React Native Notları

Meta Android Developer Professional Certificate - React Native Notları

Tarih: 16-27.12.2025

Program: Meta Android Developer Professional Certificate

Konular: React Native temelleri, Expo kurulumu, Core Components (View, Text, ScrollView, FlatList, SectionList, TextInput), Styling, Keyboard handling, Form oluşturma

React Native Nedir?

React Native, Meta tarafından geliştirilen açık kaynaklı bir JavaScript kütüphanesi. Cross-platform native mobil uygulamalar (iOS, Android, TV, Windows) geliştirmek için kullanılıyor. JavaScript ve React bilgisiyle native uygulamalar yazabiliyorsun.

Önemli noktalar:

- Tek kod tabanı ile hem iOS hem Android uygulaması yazabiliyorsun
- Native performans sağlıyor (JavaScript yazıyorsun ama native kod render ediliyor)
- Core components'lar var: `View`, `Text`, `TextInput`, `ScrollView` vb.

```
// Basit bir React Native bileşeni örneği
import { View, Text } from 'react-native';

function WelcomeApp() {
  return (
    <View style={{ flex: 1, justifyContent: 'center', alignItems: 'center' }}>
      <Text>Welcome to React Native!</Text>
    </View>
  );
}
```

Mobil Geliştirme Türleri

Native Development

- Platforma özel: iOS için Swift/Objective-C, Android için Java/Kotlin
- Farklı ekipler, farklı kod tabanları gerekiyor

- Yüksek performans, tam native erişim

Cross-Platform Native (React Native)

- Tek kod tabanı, hem iOS hem Android
- JavaScript + React ile geliştirme
- Native component'ları wrap ediyor

Hybrid Apps

- Web view tabanlı (Cordova, Ionic)
- Native özelliklere plugin'lerle erişim
- Performans daha düşük, tam native değil

Expo ile Geliştirme Ortamı Kurulumu

Ön Koşullar

- Node.js (en son sürüm)
- IDE (VS Code önerilir)
- Emulator (iOS için Mac gerekiyor) veya fiziksel cihaz

Kurulum Adımları

```
npx create-expo-app ProjeAdi
cd ProjeAdi
npm start
```

Expo Avantajları

- **Kolay kurulum:** Native kod bilgisi gerekmiyor
- **OTA Updates:** App Store onayı beklemeden güncelleme
- **Built-in APIs:** Camera, location, notifications hazır geliyor

Expo Dezavantajları

- Tüm native API'ları desteklemiyor
- Custom native module eklemek zor
- App boyutu daha büyük olabiliyor
- **Eject seçeneği var:** Expo'dan çıkıp vanilla React Native'e geçebilirsiniz

React Native Component Türleri

1. Core Components

React Native paketiyle gelen temel component'lar:

- `View`: Container, div gibi
- `Text`: Tüm metinler bu component içinde olmalı
- `ScrollView`: Kaydırılabilir liste
- `TextInput`: Input alanı
- `Image`: Resim gösterme

```
import { View, Text, StyleSheet } from 'react-native';

function MyComponent() {
  return (
    <View style={styles.container}>
      <Text style={styles.text}>Merhaba!</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff'
  },
  text: {
    fontSize: 20,
    color: 'black'
  }
});
```

2. Community Components

React Native community tarafından geliştirilen paketler:

- React Navigation (sayfalar arası geçiş)
- React Native Maps
- React Native Video

3. Custom Native Components

Kendi native component'ını oluşturabilirsin (Swift, Java, Kotlin ile)

View Component

UI'nin temel building block'u. Div gibi düşünebilirsin.

Özellikleri:

- Container olarak kullanılır
- Styling, flexbox, touch handling destekler
- İç içe view'lar kullanabilirsin
- `flex` property ile parent'a göre alan kaplama oranı belirleyebilirsin

```
<View style={{ flex: 1, backgroundColor: 'green' }}>
  <View style={{ flex: 0.3, backgroundColor: 'yellow' }}>
    {/* Bu view parent'ın %30'unu kaplar */}
  </View>
</View>
```

Text Component

ÖNEMLİ: Tüm metinler `<Text>` component'ı içinde olmalı!

Özellikleri:

- Styling destekler
- Nesting yapabilirsin (child text parent'ın stilini inherit eder)
- `numberOfLines` prop'u ile satır sayısı sınırlayabilirsin

```
<Text style={{ fontSize: 20, color: 'black' }}>
  Welcome to
  <Text style={{ fontWeight: 'bold' }}>Little Lemon</Text>
</Text>
```

ScrollView Component

Kaydırılabilir liste yapmak için. **DİKKAT:** Tüm child'ları aynı anda render eder, bu yüzden büyük listelerde performans sorunu olabilir.

```
<View style={{ flex: 1 }}>
  <ScrollView
    style={{ padding: 40, backgroundColor: 'green' }}
    showsVerticalScrollIndicator={true}
    indicatorStyle="white"
  >
    <Text>Item 1</Text>
    <Text>Item 2</Text>
    {/* ... birçok item */}
  </ScrollView>
</View>
```

Prop'lar:

- `horizontal={true}`: Yatay kaydırma
- `keyboardDismissMode="on-drag"`: Klavye kaydırınca kapanır
- `indicatorStyle`: Scroll indicator rengi

StyleSheet API

Inline styling yerine StyleSheet kullanmak kod okunabilirliği için çok daha iyi.

```
import { StyleSheet } from 'react-native';

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
    backgroundColor: '#fff'
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    color: '#333'
  }
});

// Kullanımı
<View style={styles.container}>
  <Text style={styles.title}>Başlık</Text>
</View>
```

Avantajları:

- Kod daha temiz ve okunabilir
- Style'ları reuse edebilirsin
- Performans optimizasyonu sağlar

FlatList Component

Büyük listeler için ScrollView'den daha performanslı. **Lazy loading** yapar: sadece ekranda görünen ve görünecek olan item'ları render eder.

```
import { FlatList } from 'react-native';

const menuItems = [
```

```

    { id: '1', name: 'Hummus' },
    { id: '2', name: 'Falafel' },
    // ... daha fazla item
  ];

function MenuItems() {
  const renderItem = ({ item }) => (
    <View style={styles.itemContainer}>
      <Text>{item.name}</Text>
    </View>
  );

  return (
    <FlatList
      data={menuItems}
      renderItem={renderItem}
      keyExtractor={item => item.id}
      ListHeaderComponent={<Text>View Menu</Text>}
      ItemSeparatorComponent={() => <View style={styles.separator} />}
      ListFooterComponent={<Text>© 2023 Little Lemon</Text>}
    />
  );
}

```

Gerekli Prop'lar:

- `data`: Item'ların array'i
- `renderItem`: Her item'ı nasıl render edeceğinin fonksiyonu

Faydalı Prop'lar:

- `keyExtractor`: Her item için unique key
- `ListHeaderComponent`: Listenin başına component
- `ListFooterComponent`: Listenin sonuna component
- `ItemSeparatorComponent`: Item'lar arası separator

SectionList Component

FlatList gibi ama section'lara ayırabiliyorsun. Menü kategorileri gibi düzenlemeler için ideal.

```

import { SectionList } from 'react-native';

const menuData = [
  {

```

```

    title: 'Appetizers',
    data: ['Hummus', 'Falafel', 'Eggplant Salad']
  },
  {
    title: 'Main Dishes',
    data: ['Grilled Fish', 'Lamb Kebabs', 'Chicken Shawarma']
  }
];

function Menu() {
  const renderSectionHeader = ({ section }) => (
    <Text style={styles.sectionHeader}>{section.title}</Text>
  );

  const renderItem = ({ item }) => (
    <Text style={styles.item}>{item}</Text>
  );

  return (
    <SectionList
      sections={menuData}
      renderItem={renderItem}
      renderSectionHeader={renderSectionHeader}
      keyExtractor={({item, index}) => item + index}
    />
  );
}

```

iOS vs Android Farkı: iOS'ta section header'lar scroll ederken üstte kalır (sticky), Android'de kalmaz.

TextInput Component

Form oluşturmak için kullanılan input component'ı. Native klavye açar.

```

import { useState } from 'react';
import { TextInput, View } from 'react-native';

function FeedbackForm() {
  const [firstName, setFirstName] = useState('');
  const [message, setMessage] = useState('');

  return (
    <View>
      <TextInput

```

```

        value={firstName}
        onChangeText={setFirstName}
        placeholder="First Name"
        style={styles.input}
        onFocus={() => console.log('Focused!')}
        onBlur={() => console.log('Blurred!')}
    />

    <TextInput
        value={message}
        onChangeText={setMessage}
        placeholder="Leave feedback"
        multiline={true}
        maxLength={250}
        style={[styles.input, styles.multilineInput]}
    />
</View>
);
}

```

Useful Props:

- `placeholder`: Input boşken gözüken text
- `value`: Input değeri
- `onChangeText`: Değer değişince tetiklenen fonksiyon
- `multiline`: Çoklu satır
- `maxLength`: Maksimum karakter
- `keyboardType`: Klavye türü (`default`, `number-pad`, `email-address`, `phone-pad`)
- `secureTextEntry`: Password field için (true/false)
- `onFocus` / `onBlur`: Focus olayları
- `clearButtonMode` (iOS): Clear butonu görünümü (`never`, `while-editing`, `unless-editing`, `always`)

Keyboard Handling

Mobilde klavye yönetimi önemli. Sınırlı ekran alanında klavye elementleri kapatabilir.

KeyboardDismissMode

```

<ScrollView keyboardDismissMode="on-drag">
    { /* Input'lar burada */ }
</ScrollView>

```


- `'none'`: Kaydırınca klavye kapanmaz (default)
- `'on-drag'`: Kaydırma başlayınca klavye kapanır
- `'interactive'` (iOS): Interactive kapanma

KeyboardAvoidingView

Klavye açılınca view'ı yukarı kaydırır, input'ların görünür kalmasını sağlar.

```
import { KeyboardAvoidingView, Platform } from 'react-native';

<KeyboardAvoidingView
  behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
  style={styles.container}
>
  <ScrollView>
    {/* Form içeriği */}
  </ScrollView>
</KeyboardAvoidingView>
```

Behavior değerleri:

- `'height'`: View'ın height'ını değiştirir
- `'position'`: Position değiştirir
- `'padding'`: Bottom padding ekler (iOS için önerilen)

Özel Notlarım: Button ve Touch Components

Button Component

```
import { Button } from 'react-native';

<Button
  title="Press Me"
  onPress={() => alert('Pressed!')}
  disabled={false}
  color="#841584"
/>
```

Özellikleri:

- `title`: Buton üzerindeki yazı (zorunlu)
- `onPress`: Tıklanınca çalışacak fonksiyon (arrow function ile)
- `disabled`: Butonu pasif yapar
- `color`: Buton rengi (Android'de background, iOS'te text rengi)

DİKKAT: Button çok sınırlı customize edilebilir. Daha fazla özelleştirme için TouchableOpacity veya Pressable kullan.

TouchableOpacity

Herhangi bir element'i tıklanabilir yapar. Opaklık efekti verir.

```
import { TouchableOpacity, Text, Image } from 'react-native';

<TouchableOpacity
  onPress={() => console.log('Tıklandı!')}
  onLongPress={() => console.log('Uzun basıldı!')}
  activeOpacity={0.7}
  style={styles.button}
>
  <Text>Tıkla Beni</Text>
  <Image source={require('./icon.png')} />
</TouchableOpacity>
```

İçine birden fazla component koyabilirsin: Text, Image, View vb.

Prop'ları:

- `onPress`: Normal tıklama
- `onLongPress`: Uzun basma
- `activeOpacity`: Tıklanma anındaki opaklık (0-1)
- `style`: Her türlü styling

TouchableHighlight

TouchableOpacity'e benzer ama arkaplan rengi değişimi efekti var.

```
import { TouchableHighlight, View, Text } from 'react-native';

<TouchableHighlight
  onPress={() => alert('Tıklandı')}
  underlayColor="#DDDDDD"
  style={styles.button}
>
  <View style={{ padding: 20 }}>
    <Text>Button Text</Text>
    <Text>İkinci metin</Text>
  </View>
</TouchableHighlight>
```

ÖNEMLİ: TouchableHighlight içine birden fazla component koyacaksan mutlaka `<View>` ile wrap et! Çünkü tek child kabul eder.

Prop'lar:

- `underlayColor`: Tıklanınca çıkan renk
- `onPress`: Tıklanma fonksiyonu
- `style`: Styling

Dimensions API

Cihaz ekran boyutlarını almak için:

```
import { Dimensions } from 'react-native';

const windowHeight = Dimensions.get('window').width;
const windowHeight = Dimensions.get('window').height;

// Veya destructring ile:
const { width, height } = Dimensions.get('window');

// Responsive tasarım için kullanabilirsin:
const responsiveFontSize = width * 0.04; // ekran genişliğinin %4'ü
```

Not: Bu notlar Meta Android Developer Professional Certificate programının React Native derslerinden alınmıştır.